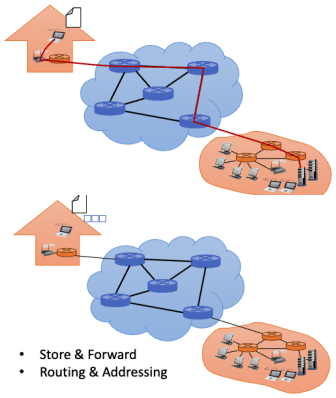
Lecture 1: Introduction

www, email, ssh, ftp, rdp and ntp are services that run over the internet
internet: network of connected computing devices
- infrastructure connecting hosts/end systems tgt, allow hosts to communicate
- network edge: end hosts, servers - network core: ISPs, routers

http: hypertext transfer protocol (secure)
Protocols define the format & order of messages exchanged among network entities and actions taken receiving or sending the messages
Hosts connect to access network over different physical media (cable):
- Guided media (signals propagate in solid media eg fiber)
- Unguided media (signals propagate freely eg Wi-Fi, cellular)

The Network Core: A mesh of interconnected routers
How data is transmitted through the network:



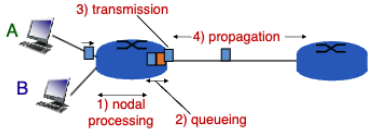
1. **Circuit switching**
End-end resources allocated to and reserved for "call" between source & destination
- call setup required
- circuit-like performance
- circuit segment idle if not used by call
- commonly used in traditional telephone networks
- divide link bandwidth into "pieces" (frequency, time division)

2. **Packet Switching (internet uses)**
Host sending function:
- breaks application msg into smaller chunks = packets of L length bits
- transmits packets onto link at transmission rate R (link transmission rate/capacity/bandwidth)

packet transmission delay = time needed to transmit L-bit packet into link = L (bits) / R (bits/sec)

Store-and-forward:
- pkts are passed from one router to the next, across links on path from src to dest
- s-a-f: entire pkt must arrive at router before it can be transmitted on the next link
- end-to-end delay = 2*L/R (assume no other delay, else refer to srcs of pkt delay)

Routers determine src-dest route taken by packets.
Addressing: each packet needs to carry src and dest info
- pkts queue in router buffers
- queue has finite capacity
- packet arriving to full queue will be dropped
- lost pkt may be retransmitted by prev node, by src host or not at all

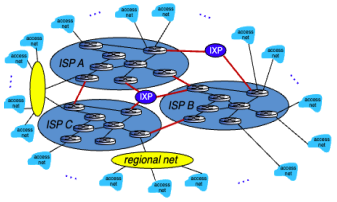


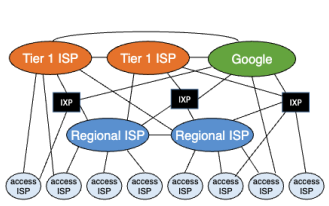
srcs of pkt delay:
1. Processing: check bit errors, determine output link, typically < msec
2. Queuing: time waiting in q for transmission, depends on congestion lvl of router
3. Transmission delay: Packet length (L bits) / link bandwidth (R bps)
4. Propagation delay: length of physical link d / propagation speed s (~2×10^8 m/s)

Throughput: how many bits can be transmitted per unit time

Circuit vs Packet
setup/teardown req vs no setup/teardown
resources are reserved vs shared on demand resources
guaranteed service vs best effort service

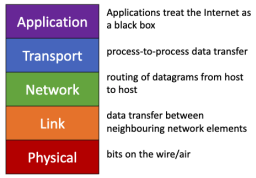
Access ISP (Internet Service Providers) are connected to global ISP





NIC: ip address and internet naming
ISOC: provide leadership in internet related standards, education and policy
IAB: authority to issue and update technical standards regarding internet protocols
IETF: protocol engineering, development and standardisation arm of IAB
Internet standards published as RFCs (request for comments)

Protocol layers:



Lecture 2: Application Layer

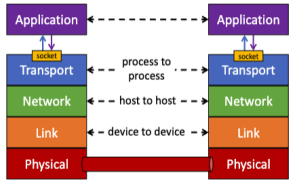
Client-server Architecture:
- server: waits for incoming requests, provides requested service to client
- client: initiates contact w/ server, requests service, implemented in browser

Peer-to-peer Architecture:
- no always-on server, arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
- highly scalable but difficult to manage

Hybrid (eg instant messaging)

Transport Services an app needs:
1. Data integrity (eg file transfer) 2. Throughput (eg multimedia) 3. Timing (eg online interactive games) 4. Security

App-layer protocols define:
1. Type of messages exchanged 2. Rules (how and when apps send & respond) 3. Msg syntax 4. Msg semantics 5. Open protocols 6. Proprietary protocols

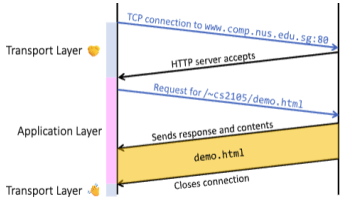


Identifying recipient
1. Globally unique address (IP)
- identifies host
- IPv4 32 bit number (dotted decimal notation eg 192.168.0.1)
- IPv6 128 bit (hexadecimal notation)

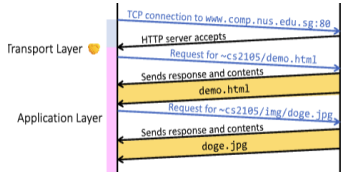
2. Locally unique address (Port no.)
- identifies process
- 16 bit number (1 to 65535)
- 1 to 1023 reserved (IANA assigned)

Internet Transport Protocols
1. TCP (Transmission Control Protocol): connection oriented, flow controlled, congestion controlled, reliable
- time needed to setup a connection - prevents sender from flooding receiver - mitigates congestion - guarantees delivery of data (no guarantees on thruput/delay)
2. UDP (User Datagram Protocol): unreliable data transfer

URL Uniform Resource Locator: explains how to locate resources
A webpage is a HTML file with several other objects
Web objects are addressable by a URL
Web resources are requested and received using HTTP (uses TCP)

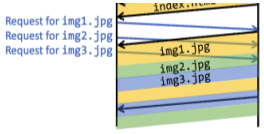


HTTP 1.0 (non persistent)
Whole process repeats for each object in html file
RTT: Round trip time → time for a pkt to travel client to server and back
HTTP response time:
- 1 RTT to establish TCP
- 1 RTT for HTTP req and first few bytes of HTTP res to return
- file transmission time (ftt)
- total = 2*RTT + ftt



HTTP 1.1 (persistent / keep alive)
- allows multiple requests to be sent over the same connection
- supports pipelining → multiple req to be sent at once w/o waiting for prev res
- as little as 1 RTT for all referenced obj.

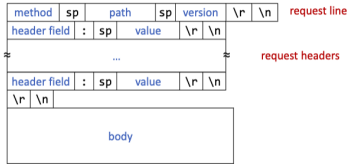
Non-persistent issues: OS overhead for each TCP connection, browsers often open parallel TCP connections to fetch referenced objects.



HTTP 2: Multiplexing
- res can come back in any order, even partially

HTTP request format

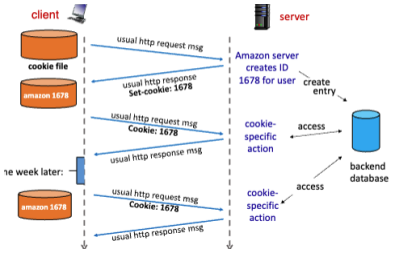
```
GET /~cs2105/demo.html
HTTP/1.1 \r\n
Host: www.comp.nus.edu/sg
\r\n
User-Agent: Mozilla/5.0 \r\n
Connection: close \r\n
```




```
HTTP/1.1 200 OK \r\n
Date: Wed, 01 Jul 2015 08:47:52 GMT \r\n
Server: Apache/2.4.6 (Unix) OpenSSL/1.0.1m \r\n
Accept-Ranges: bytes \r\n
Connection: Keep-Alive \r\n
Content-Length: 73 \r\n
Content-Type: text/html \r\n
Keep-Alive: timeout=5, max=100 \r\n
\r\n
<!DOCTYPE html>
<html lang="en">
...
```

Data, e.g. requested HTML file

HTTP status codes:
200 OK: request successful, requested object follows
301 Moved Permanently: new loc to follow
304 Not Modified: Object has not changed since specified date/time
403 Forbidden: Server declines to show the webpage
404 Not Found: Requested object not found
500 Internet Server Error: Unspecified error



HTTP designed to be stateless. (server maintains no info abt past client req)
Cookies used to maintain state. (eg shopping carrots, login acct)
Cookie: http msgs carry "state"
- cookie header field of HTTP req/res msgs
- cookie file kept on users host, managed by user's browser
- backend database at Web site

Conditional GET (to check if resource has changed): dont send object if client cache has up-to-date cached version → specify date of cached copy in HTTP req → server response contains no obj if cached copy is up to date (304 not modified)

DNS (Domain Name System): translates btw host name & IP add (runs over UDP/53)
2 ways to identify host: hostname (www.comp.nus.edu.sg) or IP address
Client must carry out DNS query to determine IP address corresponding to server name prior to connection
Mapping between host names and IP addresses are stored as Resource Records (RR) → RR format: `<name, value, type, TTL>`
DNS stores RR in distributed databases implemented in a hierarchy of many name servers. There are 13 root name servers worldwide
- TLD (top level domain) servers: .com, .org, .net, .edu, and all TL country domains
- Authoritative servers: organisations own DNS servers
- Local DNS server: does not strictly belong to hierarchy, each ISP has a local DNS server, when host makes DNS query, it is sent to its local DNS server

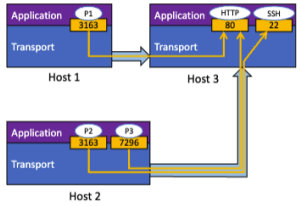
Once a name server learns a mapping, it is cached (out of date/expire(TTL))
If host changes IP add, might not propagate glocally until all cache expires

Lecture 3: Socket Programming

Processes: top level execution container, independent memory space
Threads: runs in a process, shared same memory
Apps run in hosts as processes. Within the same host, 2 processes communicate using inter-process communication. Processes in diff hosts communicate by exchanging msgs. Port number identifies the process on a host via a socket.

Standard port numbers: HTTP server - 80, POP server - 25, WoW - 3724

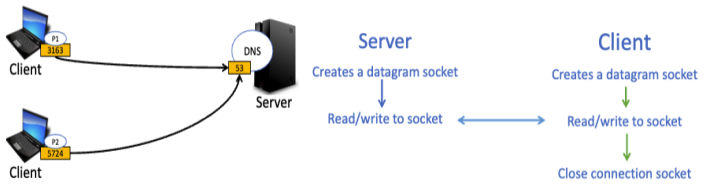
Internet Messages
Deliver pkt to the right host: IP address of host
Dispatch pkt to the right process in the host: port number of socket



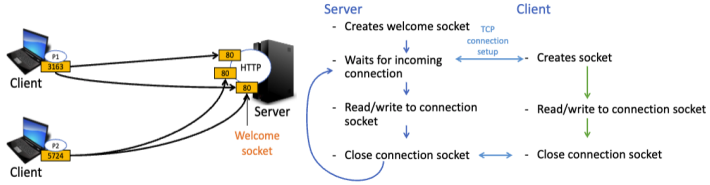
Sockets: abstraction interface between processes and transport layer protocols
- process sends/receives msgs to/from socket (a set of API calls)

2 types of sockets
- stream socket (TCP socket), uses TCP → connection oriented, reliable
- datagram socket (UDP socket), uses UDP → connection-less, unreliable

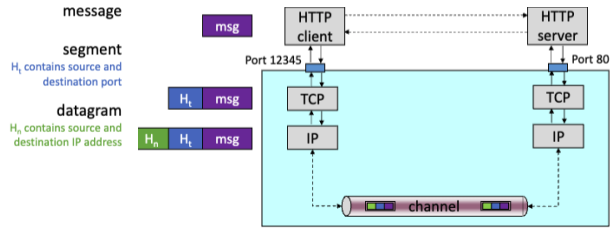
UDP
- data sent as datagrams (pkts)
- only one socket needed, communication with anyone thru same socket
- application creates each pkt, specifies dest IP add + port, OS attaches src IP + port
- receiver identifies sender by extracting src IP + port from received pkt



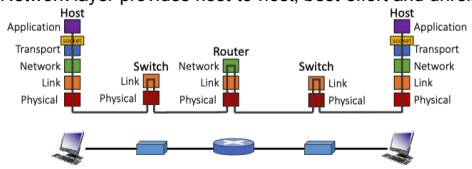
TCP
- process establishes a connection to another process, data flows btwn the processes in continuous streams. Sending process doesnt need to attach dest UP add and port no to the bytes in each sending attempt as the connection has been established.
- TCP server: creates welcome/listening socket, when contacted by client, forks a new socket for server process to communicate with client
- TCP client: creates a socket to establish connection with server, multiple sockets for multiple connections
- every connection has its own socket instance



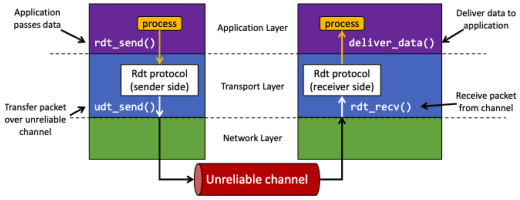
Lecture 4: Reliable Protocols
Transport layer resides on end hosts, process-to-process communication
Sender side: breaks app message into segments, passes them to network layer
Receiver side: reassembles segments into message, passes it to app layer
Packet switches in between only check dest IP add to decide routing



Transport layer resides on end hosts and provides process-to-process communication
Network layer provides host-to-host, best-effort and unreliable communication



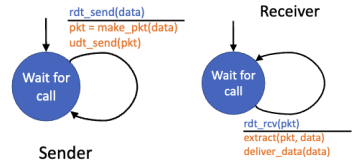
Underlying network may corrupt, drop, reorder packets and/or deliver packets after an arbitrarily long delay. End-to-end reliable transport service should guarantee packets delivery and correctness, delivering pkts in same order they are sent



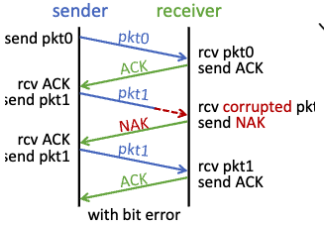
Reliable Delivery Transfer (rdt)

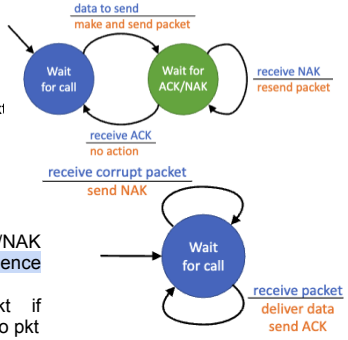
Finite State Machine: used to describe sender and receiver of protocol

rdt 1.0: assume underlying channel is reliable

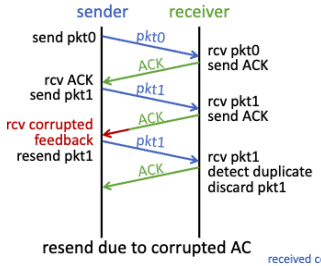


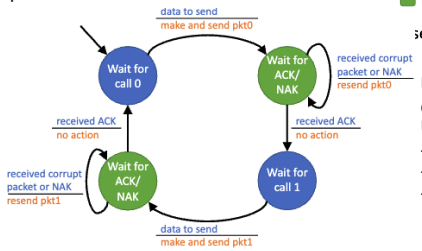
rdt 2.0: assume underlying channel may flip bits in packets
Receiver may use checksum to detect bit errors
Recovery: sender retransmits packet on receipt of NAK
Stop-and-wait protocol: send one packet at a same, wait for res



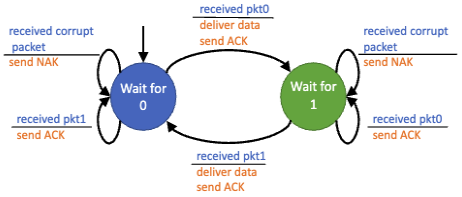


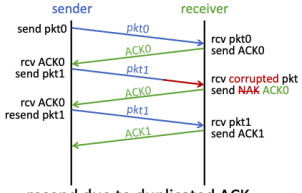
flaw: sender doesnt know if ACK/NAK corrupted → rdt 2.1: add sequence number to handle duplicates
- sender retransmit current pkt if ACK/NAK garbled, and add seq no. to pkt
- receiver discards duplicate pkt



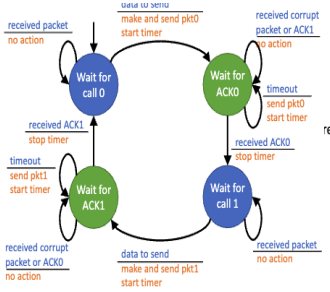


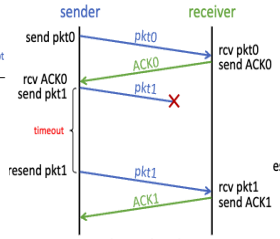
rdt 2.2: replace NAK with ACK of last correctly received pkt
- receiver explicitly include seq # of pkt being ACKed



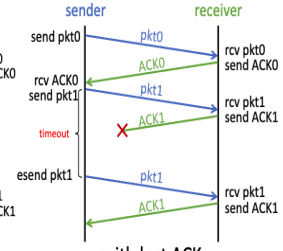


rdt 3.0: assume packet can be lost or corrupted, may flip bits, may incur arbitrarily long pkt delay
-- to handle pkt loss: sender waits reasonable amt of time for ACK, retransmit if no ACK received til timeout



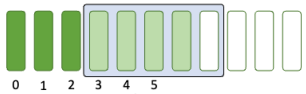


with packet loss

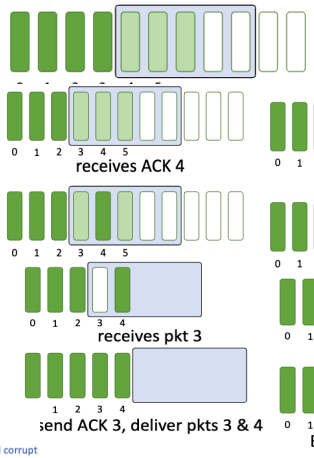


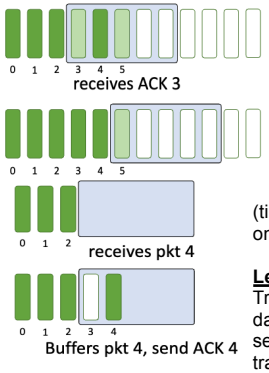
with lost ACK

Utilisation: time spent sending / total time
Problem with stop-and-wait: $U_{sender} = d_{trans} / (d_{trans} + RTT)$
Solution: Pipelining → sender allows multiple in flight, yet to be ACKed pkts
- range of seq numbers must increase + buffering at sender and/or receiver



1. Go-Back-N: Cumulative ACK → ACK n means all pkts <= n received
⇐ Sender
- sliding window of size n keeps track of n unACKed pkts, sets a timer for oldest unACKed pkt. On timeout retransmits all pkts.
- can only have up to n unACKed pkts
Receiver: if receive out of order pkt, discard and resend ACK (eg have pkts 0-3, send ACK3, receive pkt 6, discard, resend ACK3)

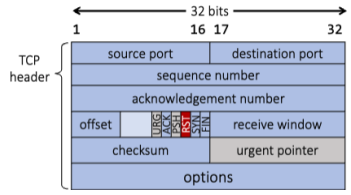


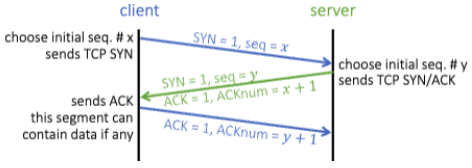


2. Selected Repeat: one timer per packet, receiver needs a buffer
- receiver individually ACKs all correctly received pkts and buffers out of order pkts
- sender maintains timer for each unACKed pkt (timer expires → transmit only that unACKed pkt)

Lecture 5: UDP & TCP
Transport-layer multiplexing: data from multiple sockets sent using only 1 transmission channel.

UDP benefits: no connection setup, no connection state, small header size, no congestion control UDP header: src port, dest post, length, checksum
UDP checksum: to detect errors in transmitted segment (only single bit flips)
- sender compute checksum value, include into UDP checksum field
- receiver compute checksum value and compare
- receiver should receive all 1s in checksum



TCP socket identified by 4-tuple: (srcIPAddr, srcPort, destIPAddr, destPort)
- seq. number: first byte of data in the segment (byte number)
- ack. number: seq # of next byte of data expected (TCP only ACKs up to themissing byte)
- how large should a segment be: Maximum Segment Size (MSS)
Key: Timeout > RTT

- $RTT_{Est} = (⅞)RTT_{Est} + (⅛)RTT_{Sample}$ - $RTT_{deviation} = ¾\ RTT_{dev} + ¼\ |\ RTT_S - RTT_{Est}\ |$
Retransmission Time Out (RTO) = $RTT_{Est} + 4 \times RTT_{dev}$
TCP Fast Retransmission: if 3 dupe ACKs, resend segment immediately
Half open connections: SYN flooding, SYN/ACK flooding (overwhelm network)
Closing connection: send segments w/ FIN bit set (cannot send but can rcv data aft)
Receive window: tells sender how much data they can send (0 means full)